

Hyperlocal Severe-Weather Nowcasting

Predicting severe thunderstorms, cloudbursts and flash floods 1-6 h ahead, at 0.1° resolution, with minutes of processing latency instead of hours.

The problem

Operational numerical weather prediction (NWP) is skilful at the synoptic scale but weak exactly where it matters most for disaster response: small, fast, convective events in the 2-6 hour window. Two gaps drive the damage.

- **Scale.** A cloudburst is a 5-10 km phenomenon lasting an hour or two. Operational guidance is issued on grids far coarser than the event itself, so the hazard is averaged away before a forecaster ever sees it.
- **Latency.** End-to-end pipelines take hours from observation to actionable product. For a hazard whose warning window is 2-6 hours, multi-hour latency consumes most of the lead time it was meant to provide.

The consequence is concrete: the 2-3 May 2018 north-India thunderstorm and dust-storm outbreak killed roughly 125 people across Delhi NCR, Uttar Pradesh and Rajasthan. That event is the demonstration case this system is built and evaluated against.

What we are building

A nowcasting system that pairs NWP fields with fast-updating satellite observations and feeds them to a spatiotemporal model, producing calibrated probability maps per hazard per lead hour, with the reasoning behind each forecast exposed to the operator and alerts emitted in a standard format.

Hazards	thunderstorm, cloudburst, flash flood
Lead times	1-6 h, hourly steps
Analysis grid	0.1° (~11 km), 38 x 49 cells
Pilot region	Uttarakhand and Delhi NCR — N 31.5, S 27.8, W 76.3, E 81.1
Vertical levels	1000, 925, 850, 700, 500, 300, 250 hPa
Model input	20 channels x 12 time steps x 38 x 49
Model output	3 hazards x 6 lead hours x 38 x 49 probability maps

Languages, and why each one

Three languages, each chosen for a job the others do worse.

Layer	Technology	Why this choice
Python	the whole data and model pipeline	The scientific stack is the reason. xarray and Zarr handle labelled N-dimensional weather data natively; MetPy implements the convective diagnostics (CAPE, CIN, lifted index) to meteorological standards, so we do not re-derive thermodynamics by hand; PyTorch trains the model. Rewriting any of that in a faster language would cost months and forfeit correctness the domain libraries already guarantee.

Layer	Technology	Why this choice
JavaScript (React)	the operator console	The console is a live map with GPU-drawn probability layers. MapLibre GL and deck.gl have no serious equivalent outside JavaScript, and the browser is the only runtime a district control room can open without installing anything.
SQL (PostGIS)	spatial and time-series storage	Alert areas are polygons; forecasts are a time series. Both are database problems with mature answers -- PostGIS for geometry, TimescaleDB for append-heavy history -- not problems to solve in application code.

On Python and speed: the usual objection is that Python is slow. It is not the bottleneck here. The numerical work runs inside compiled NumPy and PyTorch kernels, and the operational budget is dominated by network I/O and inference, not interpreter overhead. Where a hot loop did threaten the budget, the fix was vectorisation, not a change of language.

Why this is different

- **It targets the regime the benchmarks skip.** BharatBench, the established Indian ML-weather benchmark, operates at 1.08° and 6-hourly sampling for 3-5 day forecasts. We work at 0.1° and hourly steps for 1-6 hour leads — the scale at which cloudbursts physically exist.
- **Lead signals, not just fields.** The model is fed engineered precursor channels — cloud-top cooling rate, water-vapour tendency, low-level convergence, shear, CAPE and CIN — rather than raw variables alone, so it learns from the quantities that actually precede convective initiation.
- **Terrain is a first-class input.** Flow accumulation, flow direction and height above nearest drainage are computed once from a 30 m DEM and routed into the flash-flood head, so the model distinguishes rain that runs off from rain that pools.
- **One backbone, three hazards.** A shared spatiotemporal encoder with three task heads lets correlated hazards reinforce each other instead of being modelled as three unrelated problems.
- **Explainability is part of the product.** Every forecast surfaces its ranked drivers. A duty officer ordering an evacuation needs to know why, not just what.
- **Calibrated, not just confident.** Uncertainty is estimated and calibrated, so a stated 70 per cent means roughly seven events in ten.

1. Frontend

An operational console for a duty forecaster, not a dashboard for a demo. The design question was what someone at 3 a.m. must answer in five seconds: where is the hazard, how severe, how confident, and what action follows.

Status: built and running on mock data.

Stack

Layer	Technology	Why this choice
Framework	React 18 + Vite 5	Fast build and hot reload; no server-side rendering needed for a single-screen console.
Mapping	MapLibre GL 4	Open, vector-tile basemap with no proprietary key or usage ceiling.
Data overlay	deck.gl 9	GPU-rendered layers; keeps thousands of probability cells interactive.
Styling	Hand-written CSS with design tokens	A small fixed palette is easier to keep disciplined than a utility framework.

Design decisions

- **The map owns the screen.** Full-bleed layout; everything else lives behind a five-tab dock (alerts, points, drivers, model, CAP log) that collapses to a 44 px rail. All data stays one click away, none of it competes with the map.
- **Colour is reserved for severity.** No other element on screen carries chroma. Driver contributions use neutral sequential tones, because a driver weight is not a hazard level and colouring it like one would mislead the operator.
- **Motion only answers an action.** No ambient pulsing or glow; transitions respond to tab changes and expansions, and honour reduced-motion preferences.
- **Accessible by construction.** Full ARIA tablist with roving tabindex and arrow-key navigation; visible focus throughout.

Next

Replace the mock data module with live API and tile calls, and wire the timeline scrubber, driver rail and CAP panel to real model output.

2. Backend

The serving layer between trained model and console, and the component that carries the latency claim. Its target is a processing budget measured in minutes.

Status: designed, not yet implemented.

Planned stack

Layer	Technology	Why this choice
API	FastAPI + WebSocket	Async request handling; WebSocket pushes new nowcast cycles without polling.
Spatial store	PostGIS	Geometry queries for districts, basins and alert areas.
Time series	TimescaleDB	Hypertables suit append-heavy forecast and observation histories.

Layer	Technology	Why this choice
Tiles	TiTiler	Serves probability rasters as map tiles directly from the datacube.
Orchestration	Prefect	Scheduled ingestion and inference with retries and observable runs.
Alerting	CAP 1.2	Common Alerting Protocol is the format Indian agencies already consume.

How the latency target is approached

- Incremental ingestion — process only the newest time slice, never re-read the archive.
- Precomputed static layers — terrain routing is calculated once, offline, not per run.
- Streaming rather than batch, keeping intermediate state in memory.
- Asynchronous alert generation, decoupled from the inference path.
- GPU-accelerated inference where hardware allows.

Where the latency budget goes

The target is a processing budget measured in minutes, against pipelines that currently take hours. The budget is spent roughly as follows, and each line is an engineering decision rather than an aspiration:

Ingest the newest slice	Incremental only. The archive is never re-read; one time step is appended to the datacube.
Derive lead signals	Vectorised array operations over a 38 x 49 grid. Small by construction -- the pilot domain is 1,862 cells, not a national grid.
Terrain context	Zero cost at run time. Flow accumulation, flow direction and HAND are precomputed once, offline, and simply read.
Inference	A single forward pass producing all three hazards and all six lead hours at once, because the heads share one backbone.
Publish and alert	Probability rasters written as map tiles; CAP alerts generated asynchronously so alerting never blocks the forecast path.

Two design choices do most of the work. The multi-task backbone means three hazards cost one forward pass rather than three. And precomputing terrain routing moves the most expensive geospatial work out of the operational path entirely -- it is the same rasters every run, so computing them per cycle would be pure waste.

An honest distinction

Two different quantities are often conflated. **Processing latency** is the time from input arriving to product published — that is what this system targets in minutes. **Data freshness** is bounded by the providers: reanalysis is months behind, merged satellite rainfall runs about four hours behind, and only direct geostationary imagery updates every 15-30 minutes. The operational design therefore leans on satellite for the fast-updating channel and bias-corrected NWP as the slower background. Claiming minutes of end-to-end freshness would not be truthful.

3. Machine learning

Status: data pipeline, feature engineering, models, training loop and uncertainty code are written and unit-tested against synthetic fixtures. Training on real data begins as the downloads complete.

Stack

Layer	Technology	Why this choice
Arrays	xarray + Zarr + dask	Labelled N-d arrays with chunked, parallel access to a datacube too large for memory.
Meteorology	MetPy	Derives CAPE, CIN, lifted index and moisture diagnostics from pressure-level profiles.
Terrain	pysheds / RichDEM, rioxarray	D8 flow routing, flow accumulation and height above nearest drainage.
Deep learning	PyTorch 2 + Lightning	Multi-task training loop with checkpointing and reproducible runs.
Baseline	LightGBM	Per-pixel gradient boosting: an accuracy floor and a fallback that always trains.
Explainability	Captum + SHAP	Attributions become the ranked driver list the console displays.

Pipeline

- **Ingestion.** Reanalysis, satellite and terrain sources are cropped, regridded and aligned onto one 0.1° grid in a Zarr datacube.
- **Feature engineering.** Raw fields become precursor channels: integrated water vapour and its tendency, cloud-top temperature drop-rate, CAPE, CIN, lifted index, low-level convergence and 0-1 km / 0-6 km shear.
- **Terrain routing.** Flow accumulation, flow direction and HAND rasters, computed once and broadcast as static channels.
- **Baseline model.** LightGBM per pixel — validates labels and targets before any deep model is trusted.
- **Main model.** A shared 3D-CNN / ConvLSTM backbone with three hazard heads, upgrading to a spatiotemporal transformer with satellite-to-NWP cross-attention.
- **Uncertainty.** Monte-Carlo dropout with probability calibration.
- **Bias correction.** Bridges the gap between training on reanalysis and running on operational analyses.

Engineering discipline

- A single configuration module is the sole source of truth for grid, bbox, levels, hazards and lead times; no module redefines them locally.
- Train, validation and test splits are disjoint date ranges, and label horizons are masked at the boundaries so future information cannot leak backwards.
- Labels are peak-normalised, with a configured occurrence threshold applied consistently wherever a hard classification is required.
- The codebase went through three independent write-review-fix rounds; every blocking finding was closed before the build was declared complete.

4. Research and data

Data sources

Layer	Technology	Why this choice
IMDAA	NCMRWF regional reanalysis, 12 km	Primary atmospheric state for training; India-specific assimilation.
MERA	NCMRWF merged rainfall, 4 km	Highest-resolution rainfall truth available for the region.
GPM IMERG	NASA, 0.1 deg, 30-minute	Independent rainfall label on exactly the analysis grid.
INSAT-3D / 3DR	ISRO via MOSDAC	Water-vapour and thermal-IR imagery: the only genuinely real-time channel.
Copernicus GLO-30	30 m elevation	Terrain routing for the flash-flood head.
ERA5	ECMWF, 0.25 deg, hourly	Self-service substitute where queued national sources cannot arrive in time.

Region-specific behaviour

The pilot deliberately spans two regimes that behave nothing alike: the Himalayan slopes of Uttarakhand, where orographic lift drives cloudbursts and steep terrain converts rain into flash floods within the hour, and the Delhi NCR plains, where convection is thermally driven and the same rainfall produces urban inundation rather than a torrent. A single set of thresholds applied to both would be wrong in one of them.

Our own label audit measured the split rather than assuming it. Across 9.07 million cell-timesteps, thunderstorm positives appear in all 1,862 grid cells with the top decile holding only 22.6 % of them -- spread evenly, as thermal convection should be. Cloudburst positives occur in just 111 cells and flash-flood positives in 177, tightly clustered on the orography. The two hazard families are not merely rarer; they live in different places.

The system encodes that in four ways:

- **Terrain as a model input.** Elevation, slope, flow accumulation and height-above-nearest-drainage are computed once from a 30 m DEM and fed to the network as static channels. The model learns the regional distinction from the terrain itself rather than from a region label pasted on top.
- **A dedicated flash-flood head.** Routed terrain context goes to the flash-flood branch specifically, so it can distinguish rain that runs downhill into a valley from rain that pools on a plain.
- **Per-region normalisation and calibration.** Statistics are fitted per region, because a 99th-percentile rainfall hour in the hills is not the same number as in the plains. Calibration is fitted the same way, so a stated 70 % means the same thing in both.
- **Region-aware thresholds.** Hazard criteria are configuration, not hard-coded constants, so each region carries the rainfall and accumulation thresholds its own climatology supports.

The practical consequence is that adding a third region is a configuration and retraining exercise, not a rewrite. The bounding box, grid and hazard criteria all live in one configuration module; nothing in the architecture is specific to these two regions.

Positioning against existing work

BharatBench established data-driven weather forecasting over India at 1.08° and 6-hourly resolution, benchmarked at 3-day and 5-day leads. IndiaWeatherBench extends regional benchmarking with probabilistic metrics. Both address medium-range forecasting. This project targets the complementary regime — convective nowcasting at 0.1° and 1-6 hour leads — where NWP skill is weakest and where the hazards that kill people actually occur.

Known limitations

- Terrain routing is computed after regriding to ~11 km cells, which under-resolves real flash-flood catchments. Routing at native 30 m resolution and aggregating afterwards is the correct fix.
- Convective diagnostics are derived from seven pressure levels; a denser profile would improve CAPE and lifted-index quality.
- Where a national source is still queued, a coarser global substitute is used. That is a stated substitution, not a silent one.

Future scope

- **Pretraining on convective archives.** SEVIR provides more than 10,000 storm events at 1 km and 5-minute sampling with aligned radar, satellite and lightning. Pretraining the backbone there and fine-tuning on Indian data should beat training from scratch on a few regional seasons.
- **Assimilating ground radar.** Doppler weather radar would add the one observation type currently missing from the input stack.
- **National rollout.** Nothing in the architecture is specific to the two pilot regions; the bounding box is configuration, not code.
- **Lightning prediction.** A fourth head, once a lightning-detection feed is available for labels.
- **Impact-based forecasting.** Combining hazard probability with population and infrastructure exposure to rank alerts by consequence rather than by intensity.
- **Edge deployment.** A distilled model running at district control rooms, so warnings survive a network outage during the event they are warning about.

Figures describing the grid, region, lead times and model shapes in this document are read directly from the project configuration at generation time.